Smart Parking Fee Management

Problem statement

Calculate parking charges using entry and exit times, vehicle categories, overnight parking, daily limits, and penalty conditions while validating time inputs.

Assumptions and rules

• Times are entered as 24-hour HH:MM on the same parking stay; if exit time is earlier than entry time, the stay crosses midnight.

• Duration is rounded up to a whole hour for billing.

• Car rate is 30 per hour and two-wheeler rate is 15 per hour.

• A daily cap of 200 for cars and 100 for two-wheelers applies per 24-hour billing day.

• An invalid vehicle type or time is rejected. A flat 100 penalty is added if the vehicle stays more than 24 hours.

Algorithm

1. Read entry time, exit time, and vehicle category.

2. Split each time into hours and minutes and validate ranges.

3. Convert times to minutes after midnight.

4. If exit is earlier than entry, add 24 hours to exit.

5. Calculate duration in minutes and round up to billed hours.

6. Choose hourly rate and daily cap from vehicle category.

7. Calculate hourly charge and apply daily cap per started 24-hour period.

8. Add the long-stay penalty when duration exceeds 24 hours.

9. Display duration and final charge.

Pseudocode

```
START\nINPUT entry, exit, vehicle_type\nVALIDATE HH:MM values\nConvert times to minutes\nIF exit < entry THEN exit = exit +
```

Flowchart

```
[Start] -> [Read entry, exit, category]\n   |\n[Validate time and category]\n   |\n[Convert times to minutes; handle midnig
```

Python solution

The program uses beginner-level Python constructs from the assigned syllabus: variables, data types, operators, conditions, loops, strings, lists, tuples, and unpacking where appropriate.

```
def to_minutes(time_text):
    parts = time_text.split(":")
    if len(parts) != 2:
        return -1
    hour = int(parts[0])
    minute = int(parts[1])
    if hour < 0 or hour > 23 or minute < 0 or minute > 59:
        return -1
    return hour * 60 + minute

entry_text = "22:30"
exit_text = "02:15"
vehicle_type = "CAR"

entry = to_minutes(entry_text)
exit_time = to_minutes(exit_text)
```

```
if entry == -1 or exit_time == -1:
    print("Invalid time")
elif vehicle_type != "CAR" and vehicle_type != "TWO_WHEELER":
    print("Invalid vehicle category")
else:
    if exit_time < entry:
        exit_time = exit_time + 1440

    duration = exit_time - entry
    billed_hours = duration // 60
    if duration % 60 != 0:
        billed_hours = billed_hours + 1

    days = duration // 1440
    if duration % 1440 != 0:
        days = days + 1

    if vehicle_type == "CAR":
        rate = 30
        daily_cap = 200
    else:
        rate = 15
        daily_cap = 100

    hourly_charge = billed_hours * rate
    maximum_charge = days * daily_cap
    if hourly_charge < maximum_charge:
        charge = hourly_charge
    else:
        charge = maximum_charge

    penalty = 0
    if duration > 1440:
        penalty = 100

    total = charge + penalty
    print("Duration in hours:", round(duration / 60, 2))
    print("Billed hours:", billed_hours)
    print("Parking charge:", charge)
    print("Penalty:", penalty)
    print("Total fee:", total)
```

Sample output

```
Duration in hours: 3.75\nBilled hours: 4\nParking charge: 120\nPenalty: 0\nTotal fee: 120
```

Explanation / observations

The sample uses a same-date overnight stay from 22:30 to 02:15. For a production system, date and time-zone handling would also be required.

Conclusion

This solution models the stated case using a clear sequence of input, processing, decisions, repetition, and output. The assumptions and sample data can be changed to test other cases.